AI for Mathematics

Week 2: Foundations of Type Theory

Ma Jiajun

School of Math, XMU / Dept. of Math, XMUM

April 14, 2026

Today's Plan

Part I – Why Types?
30 min

Programming intuition to mathematical typing

Part II – Functions and Computation
35 min

Lambda calculus, substitution, proof terms

Part III – Dependent Types and Proofs
50 min

Pi, Sigma, inductive, and identity types

Part IV – Equality, Homotopy, HoTT
45 min

Paths, transport, homotopy, early Ch. 3

These four topics form the **theoretical backbone of Lean and HoTT**.
Week 1 introduced the interface; Week 2 explains the engine.

Part I

Why Types?

You Already Know Types

Every programmer has seen this:

C

```
int x = 42;       // OK
int y = "hello"; // ERROR!
```

The compiler **rejects** the wrong type.

Lean

```
#check 42         -- 42 : Nat
#check "hello"    -- "hello" : String
#check (42 + "hi")
```

Same idea – Lean checks types too.

Open in live.lean-lang.org

The Same Intuition, Pushed Further

Type theory takes this programming intuition to the extreme:
types are not only for catching bad code;
they can express **mathematical structure and mathematical statements**.

Compiler thinking becomes foundational mathematics:
well-typed expressions are meaningful objects,
and proofs themselves become typed terms.

Types in Math: $a : A$

In type theory, we write $a : A$ to say:

> `a` is given **as an object of type** `A`.
> It is not first presented as a standalone thing and then checked for membership in a set.

So the primitive idea is not

$$\text{“here is an object } a\text{, now ask whether } a \in A\text{”}$$

but rather

$$\text{“here is } a \text{ together with its mode of being: } a : A\text{”}$$

Set-theoretic picture

Type-theoretic picture

Judgment vs Proposition

Now compare the two languages directly:

Examples

- $42 : \mathrm{Nat}$ – "42 is a natural number"
- $\mathrm{true} : \mathrm{Bool}$ – "true is a boolean"
- $[1, 2, 3] : \mathrm{List\ Nat}$ – "a list of natural numbers"

Set theory: $a \in S$ is a **proposition**

"Is $\pi \in \mathbb{N}$?" is a meaningful statement.

It may be true or false.

Type theory: $a : A$ is a **judgment**

`π : Nat` is not a false proposition inside the theory.

It is simply **ill-typed syntax** and is rejected before mathematics even begins.

Why This Matters

This difference changes the foundations completely.

In set theory

You build mathematics by:

- forming sets
- asking membership questions
- defining structure on top of sets

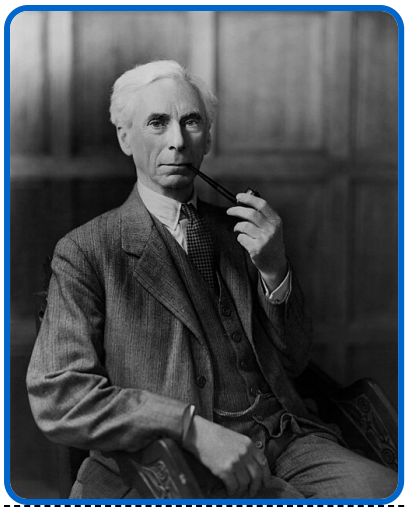
In type theory

You build mathematics by:

- declaring types
- introducing typed objects
- using formation / introduction / elimination / computation rules

That is why type theory feels closer to **syntax, construction, and computation** than classical set theory does.

Russell's Paradox



Bertrand Russell (1872–1970)

In 1901, Russell found a contradiction that shook the foundations of mathematics:

Consider "the set of all sets that don't contain themselves."

Does it contain itself?

- If **yes** → then by definition it shouldn't. Contradiction!
- If **no** → then by definition it should. Contradiction!

Why it happens: set theory lets you form *any* collection you can describe – including self-referencing ones.

How Type Theory Avoids the Paradox

Set Theory

Self-reference is **allowed** by default.

$R = \{x \mid x \notin x\}$ is a valid definition.

Fix after the fact: add axioms (ZFC) to block the bad cases.

Type Theory

Self-reference is **impossible** by design.

Types live in a **universe hierarchy**:

```
#check Nat          -- Nat : Type
#check Type         -- Type : Type 1
#check Type 1       -- Type 1 : Type 2
```

Open in live.lean-lang.org

A type at level n can only contain types at level $< n$.

The paradox is **grammatically impossible** – it is a type error, caught before you can

Two Kinds of Equality

Definitional Equality – "the compiler knows"

$$a \equiv b$$

The system can compute that both sides are the same.

```
def double (n : Nat) := n + n

-- double 3 = 6 ?
-- Lean computes: 3 + 3 → 6  ✓
example : double 3 = 6 := rfl
```

Open in live.lean-lang.org

`rfl` works because Lean **computes** both sides and they match.

Propositional Equality – "you need to prove it"

$$a = b \quad (\text{a type})$$

The system cannot just compute it – you must give a proof.

```
-- n + 0 = n ?
-- Lean cannot compute this for general n
-- example : n + 0 = n := rfl  -- ERROR!
-- You need induction to prove it.
```

Open in live.lean-lang.org

`rfl` **fails** because `n` is a variable, not a concrete number.

The Big Idea: Proofs Are Programs

In the 1930s–60s, **Haskell Curry** and **William Howard** discovered a deep connection:

Logic	Programming
True	`Unit` (a type with one value)
False	`Empty` (a type with no values)
$A \wedge B$ (and)	`A × B` (pair)
$A \to B$ (implies)	`A → B` (function)

Example: Breaking apart and reassembling

```
variable (h : A ∧ B)

#check h.1      -- A  (the left part)
```

Part II

Lambda Calculus

Alonzo Church and the λ-Calculus

Portrait from Wikipedia

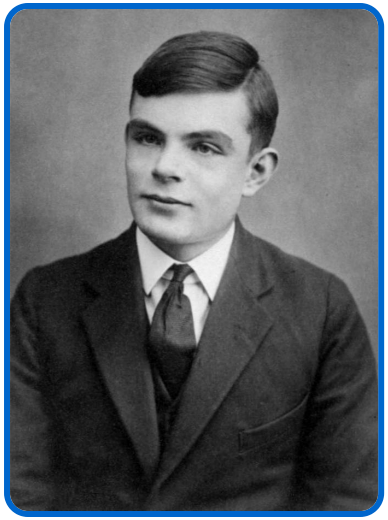
Alonzo Church (1903–1995)

American mathematician and logician. Princeton University.

- **1936:** Invented the **lambda calculus** – a formal system for defining and applying functions
- **Church-Turing thesis:** lambda calculus and Turing machines capture the same notion of computation
- Advisor of Alan Turing, Dana Scott, Stephen Kleene, and many others

> Lambda calculus is the theoretical foundation of **every functional programming language** – and of **every proof assistant** including Lean.

Why Church Matters for Computability



Alan Turing, via Wikimedia Commons

In **1936**, Church proved that there is **no general algorithm** deciding whether an arbitrary logical statement is provable.

Decision problem

Hilbert hoped for a machine that takes a statement and always answers:

- `YES` : provable
- `NO` : not provable

Church's result

No such universal procedure exists.

Some questions are **undecidable**:

- no algorithm solves every instance

The Idea: Functions as First-Class Objects

Before lambda calculus, functions in mathematics were defined by naming them:

$$f(x) = x^2 + 1$$

Church's insight: give functions a **direct notation**, without naming them.

$$\lambda x.\ x^2 + 1$$

Read: "the function that maps x to $x^2 + 1$"

λ is the abstraction operator – it binds a variable name to an expression.

$$\lambda x.\ e \quad \text{"the function that takes } x \text{ and returns } e\text{"}$$

This is exactly what we write in modern programming:

```
lambda x: x**2 + 1          # Python
fun x ⇒ x^2 + 1             # Lean 4
```

Lambda Calculus: Syntax

Lambda calculus has exactly **three** kinds of expression (called *terms*):

Variable

`x`

A name for a value

Abstraction

`λx. e`

A function definition

Application

`(f e)`

Apply f to argument e

That's it. Three rules, and yet this system is **Turing-complete** – it can compute anything that any computer can compute.

Everything else – numbers, booleans, pairs, loops – can be encoded from these three.

The Key Rule: β-Reduction

The only computation rule in lambda calculus:

$$(\lambda x.\ e)\ v \xrightarrow{\beta} e[v/x]$$

> "Apply the function $\lambda x.e$ to argument v by **substituting** v for every free x in e."

Examples:

```
(λx. x) 5 → 5 — identity function

(λx. x + 1) 4 → 4 + 1 = 5 — add one

(λx. λy. x) 3 7 → (λy. 3) 7 → 3 — constant function
```

α-Equivalence: Names Don't Matter

Two lambda terms are **α-equivalent** if they differ only in the names of bound variables.

$$\lambda x.\ x \ \equiv_\alpha \ \lambda y.\ y \ \equiv_\alpha \ \lambda z.\ z$$

$$\lambda x.\ \lambda y.\ x \ \equiv_\alpha \ \lambda a.\ \lambda b.\ a$$

This is exactly how mathematics works: in $\forall x \in \mathbb{N}, x \geq 0$, the variable x is a dummy name. We could write $\forall n \in \mathbb{N}, n \geq 0$ and mean the same thing.

Bound vs. free variables:

- x is **bound** in $\lambda x.e$ – it's the function's parameter
- y is **free** in $\lambda x.y$ – it refers to an outer definition
- Substitution $e[v/x]$ only replaces **free** occurrences of x in e

β-Reduction in Action: Watch Out for Free Variables!

β-reduction substitutes the argument for the **bound** variable. But free variables are not substituted!

```
def F (z : Nat) := z^2 + y   -- y is FREE here

-- F 3 reduces to 3^2 + y  (y is still y, not touched)
-- F 3 y reduces to 3^2 + y
```

Open in live.lean-lang.org

Key insight:

```
-- F is a function of z only
-- y is a free variable — part of the definition
example : F 3 = 3^2 + y := rfl
```

Common mistake:

```
-- Thinking F 3 replaces ALL variables
-- No! Only the bound variable z is replaced.
-- y was never bound by λ, so it stays.
```

An error occurred on this slide. Check the terminal for more information.

Simply Typed Lambda Calculus

The untyped λ-calculus has a problem: you can write nonsensical terms.

```
x x -- apply x to itself — meaningless! (λx. x x)(λx. x x) -- reduces to itself forever
— infinite loop!
```

Church's fix (1940): add types. Every variable has a type, and application requires the types to match.

Simply Typed Lambda Calculus – Typing Rules

Simply Typed Lambda Calculus (STLC):

- Every variable has a base type: Bool, Nat, …
- Function type $A \to B$: a function from A to B
- **Typing rule for abstraction:** if $x : A$ and $e : B$, then $(\lambda x{:}A.\ e) : A \to B$
- **Typing rule for application:** if $f : A \to B$ and $a : A$, then $(f\ a) : B$

Why STLC Matters

✅ Well-typed terms never loop forever (strong normalization)
✅ Well-typed terms never go wrong (type safety)
✅ Corresponds to **propositional logic** via Curry-Howard

Function Composition: Modus Ponens in Action

Function application chains propositions together – exactly like logical reasoning:

```
variable (A B C : Prop)
variable (F : A → B)   -- "If A then B"
variable (G : B → C)   -- "If B then C"
variable (a : A)        -- "A is true"

#check G (F a)   -- C  (apply G after F to a)
#check G ∘ F     -- A → C  (composed function)
```

Open in live.lean-lang.org

Logic: From $A \Rightarrow B$ and A, conclude B (modus ponens).
Programming: From `f : A → B` and `a : A`, get `f a : B`.

Curry-Howard: First Glimpse

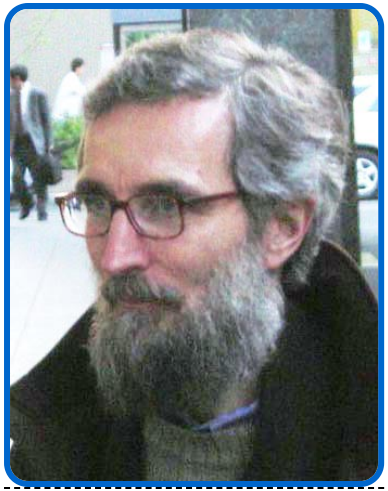
In STLC, types look exactly like **propositions** and terms look exactly like **proofs**.

Type Theory	Logic
Type A	Proposition A
Term $t : A$	Proof of A
Function type $A \to B$	Implication $A \Rightarrow B$
$\lambda x{:}A.\ e : A \to B$	"Given a proof of A, produce a proof of B"
$(f\ a) : B$	Modus ponens: from $A \Rightarrow B$ and A, conclude B

Part III

Dependent Type Theory

Per Martin-Löf



Per Martin-Löf (b. 1942)

Swedish logician, philosopher, and statistician. Professor at Stockholm University.

- **1975:** Published *Intuitionistic Type Theory* – extending STLC with dependent types
- Key insight: **allow types to depend on terms (values)**
- This makes the type system powerful enough to express all of mathematics
- Deeply influenced by **Brouwer's intuitionism** – proofs must be constructive

> Martin-Löf Type Theory (MLTT) is the direct ancestor of Lean's type system.

The Limitation of Simple Types

In STLC, function types look like $A \to B$ where B is fixed regardless of the input.

Can we write the type of "a list of exactly n elements"?

- `List Nat` – lists of natural numbers, but **length unknown**
- We want a type that says "this list has length **5**"

In simple type theory: **impossible**. The length is a runtime value; types are fixed at compile time.

The problem in concrete terms:

```
 -- Simple type
def head : List Nat → Nat := ...
-- What if the list is empty? We need a runtime check, or the function is partial.

-- What we want:
```

Dependent Types: The Core Idea

To express this, types must be allowed to **mention values**.
This is the core idea of **dependent types**.

Types That Depend on Values

Key definition: A **dependent type** is a type that is a function of a value.

Simple type

$\text{List} : \text{Type}$

One type. All lists, regardless of length.

Dependent type

$\text{Vec} : \text{Nat} \to \text{Type}$

A *family* of types, one for each natural number n.

n	Type $\mathrm{Vec}\ \mathrm{Nat}\ n$	Examples
0	length-0 lists	`[]`
1	length-1 lists	`[42]`

Π Types: Dependent Function Types

The type of a function whose **output type depends on the input value** is a **Π type**.

$$\Pi\,(x : A),\ B(x)$$

↕

Lean 4 syntax:

```
(x : A) → B x
```

Open in live.lean-lang.org

For each value `x : A`, the return type is `B x`.

Examples:

```
 -- For each n, return a list of length n
(n : Nat) → Vec Nat n

-- For each n, return a proof that n + 0 = n
(n : Nat) → n + 0 = n

-- Simple function type is a special case: B doesn't mention x
(x : Nat) → Nat   ≡   Nat → Nat
```

Σ Types: Dependent Pair Types

A **Σ type** is a pair where the **type of the second component depends on the first value**.

$$\Sigma\,(x : A),\ B(x)$$

↕

Lean 4 syntax:

```
Σ x : A, B x
```

Open in live.lean-lang.org

A pair `⟨a, b⟩` where `a : A` and `b : B a`.

```
 -- "There exists a natural number greater than 5"
example : Σ n : Nat, n > 5 := ⟨6, by norm_num⟩
--                            ↑  ↑
--                        witness  proof that 6 > 5

-- First component: the witness (n = 6)
-- Second component: a proof that 6 > 5 (whose type depends on n = 6)
```

Open in live.lean-lang.org

The Full Logic Dictionary

Every logical connective corresponds to a type constructor:

Mathematics	Type Theory	Lean syntax
Proposition P	Type P	`P : Prop`
Proof of P	Term of type P	`h : P`
$P \wedge Q$	Product type $P \times Q$	`And P Q`
$P \vee Q$	Sum type $P \oplus Q$	`Or P Q`
$P \Rightarrow Q$	Function type $P \to Q$	`P → Q`
$\forall x \in A,\ P(x)$	**Π type** $(x : A) \to P\,x$	`∀ x : A, P x`

Or, ∀, and ∃ in Action

Or (∨) – how to prove a disjunction:

```
 example (a : A) : A ∨ B := Or.inl a   -- prove left
example (b : B) : A ∨ B := Or.inr b   -- prove right
```

∀ and ∃ – concrete examples:

```
 -- "all natural numbers are non-negative"
abbrev nonneg := ∀ x : Nat, x ≥ 0
abbrev nonneg' := Π x : Nat, x ≥ 0
example : nonneg = nonneg' := rfl

-- "there exists a number ≥ 5"
abbrev geq5 := ∃ x : Nat, x ≥ 5
example : geq5 := ⟨5, Nat.le_refl 5⟩   -- witness + proof
```

Open in live.lean-lang.org

Inductive Types: Building Mathematics from Scratch

How do we define $\mathbb{N}$ in type theory? With an **inductive type**:

```
inductive Nat where
  | zero : Nat            -- 0 is a natural number
  | succ : Nat → Nat      -- if n is a natural number, so is n+1
```

Open in live.lean-lang.org

- `zero` = 0
- `succ zero` = 1
- `succ (succ zero)` = 2
- … (all natural numbers, built inductively)

Lean guarantees: every `n : Nat` is either `zero` or `succ m` for some `m : Nat`. Nothing else.

This is **Peano's axioms**, directly encoded as a type definition.

More Inductive Types

The same idea scales to all mathematical structures:

```
-- Lists
inductive List (α : Type) where
  | nil  : List α
  | cons : α → List α → List α

-- Binary trees
inductive Tree (α : Type) where
  | leaf : Tree α
  | node : Tree α → α → Tree α → Tree α
```

Open in live.lean-lang.org

Lists in Lean — equality is structural:

```
example : [1] = List.cons 1 .nil := rfl
example : [1,2] = List.cons 1 [2] := rfl
example : [1,2,3] = [1,2,3] := rfl

def onetoten := [1,2,3,4,5,6,7,8,9,10]
#print onetoten
```

Vectors: Dependently Typed Lists

```
-- Vectors (length-indexed lists) — a DEPENDENT inductive type
inductive Vec (α : Type) : Nat → Type where
  | nil  : Vec α 0
  | cons : α → Vec α n → Vec α (n + 1)

-- Construct vectors literally
def V  : Vector Nat 0 := #v[]
def V' : Vector Nat 1 := #v[42]
```

Open in live.lean-lang.org

Notice: `Vec α n` is indexed by `n : Nat`. The Lean type checker tracks the length through every operation — **at compile time**, not runtime.

Identity Types: Equality as a Type

One of the most surprising ideas in dependent type theory: **equality is itself a type**.

$$a =_A b$$

is a type. A **proof that a equals b** is a term of this type.

```
-- "2 + 2 = 4" is a proposition (a type in Prop)
#check (2 + 2 = 4)   -- Prop

-- A proof of it is a term of that type
theorem two_plus_two : 2 + 2 = 4 := by norm_num
#check two_plus_two  -- 2 + 2 = 4
```

Open in live.lean-lang.org

`rfl` – the reflexivity constructor. It proves `a = a` for any `a`:

```
example : 3 + 4 = 7 := rfl    -- Lean computes and confirms they're equal
```

Universe Hierarchy: Types of Types

Types are themselves terms – so they need types too!

`Type 2` – contains `Type 1`

⊃

`Type 1` – contains `Type`

⊃

`Type` – contains `Nat`, `Bool`, `List`, …

The Calculus of Inductive Constructions

The formal system underlying Lean is called the **Calculus of Inductive Constructions (CIC)**.

Thierry Coquand (b. 1961) – Chalmers University

1988: With Gérard Huet, developed the *Calculus of Constructions* (CoC) – combining:

- **λ-calculus** (computation)
- **Dependent Π and Σ types** (Martin-Löf)
- **Impredicative universes** (allows types to quantify over types)

CoC + Inductive Types = **CIC** → the foundation of **Coq** and **Lean**.

> Lean's type system is CIC. When Lean checks a proof, it is checking that a λ-term has the stated type in CIC.

The Big Picture: How It All Fits

λ-calculus
Church, 1936
Computation via substitution

+

Simple Types
Church, 1940
Classify terms, prevent nonsense

+

Dependent Types
Martin-Löf, 1975
Types depend on values (∀, ∃)

+

Inductive Types
Coquand et al., 1990s
Define new types from constructors

↓

Calculus of Inductive Constructions (CIC)

The foundation of Lean 4 – powerful enough to formalize all of mathematics

A Complete Example in Lean 4

Let's see all the ideas together in one proof:

```
 -- Statement: for every natural number n, n + 0 = n
-- Type:  ∀ n : Nat, n + 0 = n   (a Π type!)
-- Proof: a term of that type

theorem add_zero : ∀ n : Nat, n + 0 = n := by
  intro n           -- introduce n : Nat (λ-abstraction)
  induction n with
  | zero ⇒
    rfl             -- 0 + 0 = 0 by definition (rfl proves a = a)
  | succ n ih ⇒
    -- ih : n + 0 = n (the induction hypothesis, a term of type n + 0 = n)
    -- Goal: (n + 1) + 0 = (n + 1)
    simp [Nat.succ_add, ih]
```

Open in live.lean-lang.org

The type `∀ n : Nat, n + 0 = n` is a Π type.

Induction = pattern match on Nat's two constructors.
Same as β-reduction in λ-calculus.

Part IV

Equality, Homotopy, and Structure

Why Chapter 2 Starts with Equality

Everything in Chapter 1 was about building types and terms.

Chapter 2 asks a deeper question:

What exactly is an equality proof?
What structure does it carry?
What can we do with it?

In ordinary set-like mathematics:

- equality is yes/no
- at most one proof matters
- no geometry appears

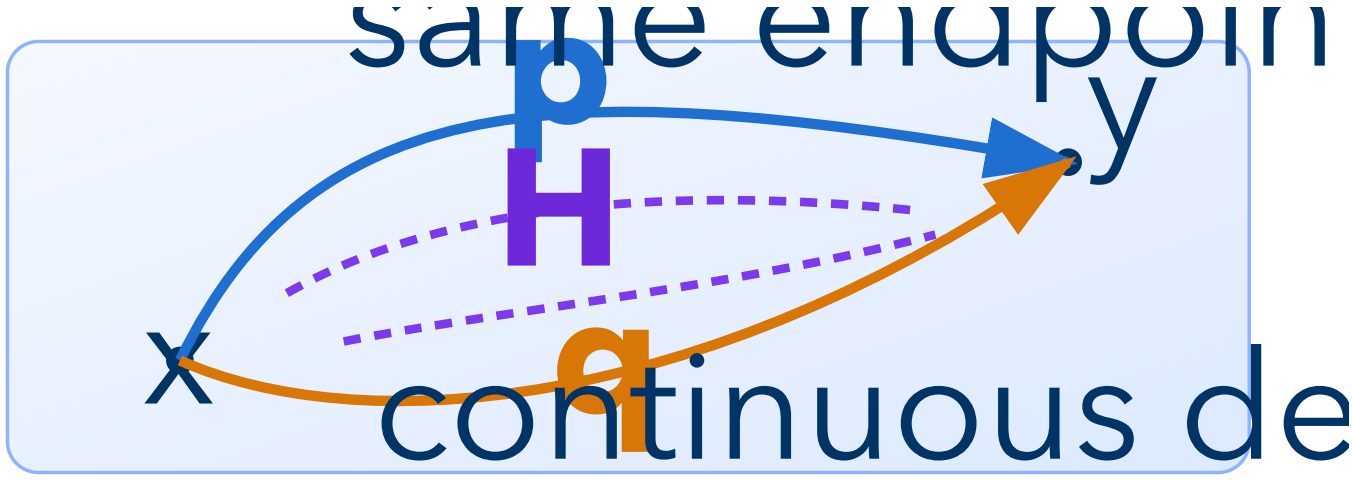
In HoTT:

- a proof of equality is a **path**
- paths can have structure
- paths between paths can also exist

What Is a Homotopy?

In topology, a path from `x` to `y` is a route in a space.

A **homotopy** between two paths is a continuous deformation from one route to the other, keeping the endpoints fixed.



HoTT takes this picture seriously: equality proofs behave like paths, and higher equalities behave like homotopies between paths.

Path Induction: The Elimination Rule for Equality

The basic rule is:

To prove something about an arbitrary path `p : x = y`,
it suffices to prove it in the special case `p = rfl`.

```
theorem my_symm_symm {α : Type}
   {x y : α} (p : x = y) :
   p.symm.symm = p := by
 cases p
 rfl
```

Open in live.lean-lang.org

This is why so many equality proofs in Lean look like `cases p; rfl`.
`cases p` is not a trick. It is the core mathematical principle.

Transport: Moving Data Along Equality

Suppose `P : A → Type` is a family of types.

If `p : x = y`, then a term of type `P x` can be moved to a term of type `P y`.

This operation is called **transport**.

It is what makes equality powerful in dependent type theory:

- not only can values be replaced by equal values,
- entire **typed objects** can move across a family.

```
-- informal shape
transport (fun z ⇒ P z) p u : P y
```

Open in live.lean-lang.org

Without transport, dependent types would be too rigid to use in mathematics.

Example: Transport for Length-Indexed Data

Imagine:

```
 α : Type
n m : Nat
p : n = m
v : Vec α n
```

Open in live.lean-lang.org

Then `p` lets us reinterpret `v` as an element of `Vec α m`.

Before transport

`v : Vec α n`

Length is indexed by `n`

After transport along `p : n = m`

`transport (fun k ⇒ Vec α k) p v : Vec α m`

Chapter 3: Not All Types Are Set-Like

HoTT classifies types by how much equality structure they contain.

Contractible

Exactly one point up to equality.

Example: `Unit`

Proposition

At most one point up to equality.

Truth values and logical statements live here.

Set

Many points may exist, but there is at most one path between two points.

General types can have **multiple different paths**, and even paths between paths. That is what makes them behave like spaces.

Beyond Chapter 1: A Glimpse of HoTT

Everything so far is **Chapter 1** of the HoTT Book. Now let's peek at what comes next – the ideas that make Homotopy Type Theory revolutionary.

Chapter 2

Equality has **structure**

Proofs of `a = b` are **paths**

Chapter 3

Not all types are equal

Sets vs. propositions vs. general types

Equality Is More Than Yes/No

In ordinary math: "$3 = 3$" is either true or false. Nothing more to say.

In HoTT: a proof of $a = b$ is a **path** – and there can be **different paths** between the same two points.

What you can do with paths:

- **Reverse**: if $p : a = b$, then $p^{-1} : b = a$
- **Compose**: if $p : a = b$ and $q : b = c$, then $p \cdot q : a = c$
- **Stand still**: rfl $: a = a$

These satisfy group-like laws (associativity, unit, inverse).

In Lean:

```
 -- Reverse a path
example (p : x = y) : y = x := p.symm

-- Compose paths
example (p : x = y) (q : y = z) :
    x = z := p.trans q

-- All proved by: cases p; rfl
-- (reduce to rfl, then it's trivial)
```

Open in live.lean-lang.org

The Shocking Example: Bool = Bool

Consider the type `Bool` and ask: in how many ways is `Bool` equal to `Bool`?

Path 1: the identity

$\mathsf{true} \mapsto \mathsf{true}$

$\mathsf{false} \mapsto \mathsf{false}$

"Bool equals Bool in the obvious way"

Path 2: the swap

$\mathsf{true} \mapsto \mathsf{false}$

$\mathsf{false} \mapsto \mathsf{true}$

"Bool equals Bool via negation"

These are **two genuinely different paths** – you can't deform one into the other.

This means the universe `Type` is **not a "set"** – it has non-trivial higher structure!

This is the content of the **univalence axiom** (Voevodsky, 2006).

Vladimir Voevodsky and Univalence

Vladimir Voevodsky (1966–2017) – Fields Medal 2002

After finding errors in published proofs (including his own), became convinced that **mathematics needs machine-checked proofs**.

Univalence axiom (2006): two types are equal iff they are equivalent.

$$(\mathsf{Bool} =_{\mathsf{Type}} \mathsf{Bool}) \;\simeq\; (\mathsf{Bool} \simeq \mathsf{Bool})$$

Led the writing of the **HoTT Book** at the Institute for Advanced Study (2012–2013).

Why Does This Matter? Classical Logic Breaks!

In classical logic, the **law of excluded middle** says: for any statement A, either A or $\neg A$.

For propositions: ✓ OK

Propositions have at most one proof.

No "symmetries" to worry about.

LEM is consistent here.

```
-- Lean's Prop has LEM built in:
open Classical in
example (P : Prop) : P ∨ ¬P := em P
```

Open in live.lean-lang.org

For general types: ✕ BREAKS

If a function must choose an element of every type, it must respect all symmetries (univalence).

For `Bool`: the choice must be a fixed point of `not`.

But `not` has **no fixed points**!

Contradiction → LEM for all types is **inconsistent**.

The n-Type Hierarchy

HoTT organizes types by how much "higher structure" they have:

General types (∞) – paths all the way down (∞-groupoids, e.g. `Type`)

⊃

Sets (0-types) – at most one path between any two points (e.g. `Nat`, `Bool`)

⊃

Propositions (-1) – at most one element (True/False)

⊃

Contractible (-2) – exactly one element

Summary: Foundations

Set Theory vs Type Theory

- Set theory starts from membership: `a ∈ A`
- Type theory starts from typed objects: `a : A`
- In type theory, objects are not first given untyped and then tested
- Typing (:) is a judgment, not a proposition
- Types prevent nonsense before proof starts

Lambda Calculus → Curry-Howard

- λ-calculus: variable, abstraction, application
- β-reduction: $(\lambda x.\, e)\, v \to e[v/x]$
- Simply typed → add dependent types → full logic

Summary: HoTT Bridge

Dependent Type Theory (Ch1)

- $\Pi = \forall$, $\Sigma = \exists$, inductive types, identity types
- transport lets equal indices move data across a family
- Universe hierarchy avoids Russell's paradox
- CIC = foundation of Lean

HoTT: Paths and Structure (Ch2–3)

- Equality proofs are **paths** with structure
- path induction reduces equalities to `rfl`
- contractible $\subset$ proposition $\subset$ set $\subset$ general type
- Bool = Bool has **two** paths (univalence)

Hands-on: Paths in Lean (5 min)

Open **live.lean-lang.org** and try these:

Paths in action:

```
 -- Check these:
#check @Eq.symm   -- reverse a path
#check @Eq.trans   -- compose paths

-- Prove by path induction:
theorem my_symm_symm {α : Type}
    {x y : α} (p : x = y) :
    p.symm.symm = p := by
  cases p; rfl

-- Your turn: prove this
theorem my_trans_symm {α : Type}
    {x y : α} (p : x = y) :
    p.trans p.symm = rfl := by
  sorry  -- replace with your proof!

-- Bonus:
example {α : Type} {x y : α} (p : x = y) :
    p.trans rfl = p := by
```

Hands-on: Transport and Bool (5 min)

Open **live.lean-lang.org** and try these:

Transport and Bool:

```
 -- Transport in a dependent family
example {α : Type} {n m : Nat}
    (p : n = m) (v : Vector α n) :
    p ▸ v = p ▸ v := by
  rfl

-- Two different endomorphisms of Bool
def boolId : Bool → Bool := id
def boolNot : Bool → Bool := not

-- They're different
#eval boolId true    -- true
#eval boolNot true   -- false

-- Your turn: prove this
example : boolId true ≠ boolNot true :=
  by decide  -- Lean can compute it!
```

Homework

1. Natural Number Game (required)

- Complete **Multiplication World** (Levels 1–9)
- Goal: practice proof by induction in Lean
- URL: https://adam.math.hhu.de/#/g/leanprover-community/NNG4

2. Path exercises in Lean (required)

- Open live.lean-lang.org
- Prove: `p.symm.trans p = rfl` (hint: `cases p; rfl`)
- Prove: `(p.trans q).trans r = p.trans (q.trans r)` (hint: `cases p; cases q; cases r; rfl`)

3. Transport exercise (required)

See You Next Week!

Week 3: Reading Proofs in Lean

We will read actual Lean proofs from Mathlib and learn how LLMs help write them